Forecasting Arbitrage in Kalshi Prediction Markets with a Spatio-Temporal Graph Attention Network

Justyn
Final Year Project: AI in Finance

Recess Week Consultation and Progress Report
September 24, 2026

Abstract

Two no-arbitrage constraints hold on the Kalshi prediction market by construction: the YES prices of a mutually exclusive, collectively exhaustive (MECE) outcome set must sum to \$1 at resolution, and prices on ordered thresholds over a common underlying must be monotone. Both are routinely violated in observed prices. Where the published literature asks whether such violations can be *detected*, this project asks whether they can be *forecast*, and whether the forecast is worth money.

The study was originally framed as a pairs-trading problem, in which statistically related contracts are identified and the spread between them traded when it diverges from its historical relationship. On supervisory advice it was redirected to the two structural constraints above, whose targets are exact and enforced at settlement rather than estimated from historical co-movement.

A spatio-temporal graph attention network is trained on five months of Kalshi data to predict the basket deviation and the monotonicity gap at horizons from two to twenty-four hours ahead. On a held-out test month, scored once with no tuning, the model attains $0.853\times$ the mean squared error of an optimal shrinkage oracle whose coefficient is fitted with hindsight on the split being scored, replicating a validation figure of $0.851\times$. An ablation attributes the large majority of that advantage to the cross-sectional graph rather than to the temporal model. The monotonicity mechanism yields a clear negative result, and the reason is instructive: its pooled reversion coefficient of $\alpha \approx 0.99$ proves to be an artifact of pairs that never trade rather than a property of market dynamics.

No strategy tested is profitable net of Kalshi's fee schedule, including an idealised static arbitrage. The report quantifies why in four independent layers: fee arithmetic, execution mode, the correlation between signal strength and trading cost, and the concentration of the model's edge in baskets too illiquid to fill. It closes with the methodological questions on which supervisory guidance is sought before the study is extended to a longer sample.

Contents

1 Summary and Open Questions for Consultation **2**

1.1 A note on how execution was modelled 3

1.2 Should static arbitrage be documented as a baseline at all? 3

1.3 How should profitability be reported when every strategy loses? 4

1.4 Is there remaining potential in the ladder mechanism? 4

2 Research Question **5**

3 Data **6**

4 Architecture **7**
4.1 Key design decisions, and why they were forced 7

5 Evaluation Protocol **8**
5.1 Notation 8
5.2 Two baselines, scored on identical positions 8
5.3 Stratification 8
5.4 PnL evaluation 8

6 Results **9**
6.1 MECE dislocation is forecastable: confirmed on held-out test 9
6.2 Ladder violations revert sharply, but only when a leg trades 9
6.3 The ladder structure cannot be used, tested four ways 10
6.4 Trading PnL: negative, for three separately quantified reasons 11
6.5 The liquidity sweep 12

7 The Model's PnL Compared to Baseline Strategies **12**
Defensible contributions 13

8 Limitations **13**

9 Code Inventory **14**

---

1 Summary and Open Questions for Consultation

I have finished the MECE and ladder evaluation and would like your view on three methodological points. Everything reported here comes from a five-month pilot (May–September 2025: two months train, two validation, one held-out test month), so all results are provisional pending a longer run.

Where I think the project stands. The model is not profitable net of costs, and I do not intend to claim otherwise, but negative net returns are the norm in the existing literature rather than an anomaly, and I believe the forecasting contribution is real and identified.

Question 1. Since you set the requirement that the model must beat baseline strategies and show PnL to be made, I would like to ask directly whether what follows, and my reasoning about it, satisfies that requirement.

Three results support the positive side of the case.

Forecasting, confirmed out of sample. On the held-out month, scored once with no tuning, the model reaches 0.853× the MSE of an optimal shrinkage oracle: a baseline deliberately handicapped in its own favour, since its α is fitted with hindsight on the split being scored. This replicates validation's 0.851×. The test month offered less mean reversion to exploit ($\alpha = 0.917$ against 0.870), so the oracle captured only 3.9% over persistence there against 6.4% on validation, yet the incremental gain held at 14.7% and 14.9%. The edge is therefore not rediscovered mean reversion: there was less of it to rediscover and the margin did not move.

The source of the edge is identified, not assumed. Removing the spatial graph entirely raises MECE MSE by 16% and collapses the margin over the oracle from 14.9% to approximately 1.2%. Roughly 92% of the advantage comes from the cross-sectional structure rather than from the temporal model, which on its own essentially reproduces the shrinkage constant and stops.

Directional skill is causally established. Against a control using the identical entry rule, events, size and execution, with only the direction replaced by a coin flip, the model wins in all three sizing regimes with every confidence interval excluding zero, and the per-contract margin grows as sizing becomes realistic ($+0.029 \to +0.040 \to +0.050$). Costs are identical to the cent, so nothing but the forecast differs.

1.1 A note on how execution was modelled

Kalshi does not retain historical order-book state, so point-in-time depth and quotes are unrecoverable. Rather than assume frictionless execution, I substituted three proxies and treated each as a sensitivity rather than as a fixed assumption.

Spread.
Measured per-leg median spreads by ticker and series from a separate spread dataset, with a flat 1¢ fallback where none was ever observed, charged at half-spread from mid on each crossing: twice per leg over a round trip.

Depth.
Position size is capped at a fixed share of the thinnest leg-side's window volume, reported over three regimes (uncapped, 25%, 10%). This is what turned the sizing assumption into the liquidity result of Section 6.5.

Maker fills.
Fills are derived from observed net order flow (a resting order fills only when flow arrives against it) rather than assumed, and compared against optimistic and volume-only fill models.

I also verified that a present price implies a trade rather than a stale quote: window volume is zero on 0.0% of observed leg-snapshots, with a median of 183 contracts, so the dislocations are quoted through under active trading rather than being an artifact of illiquidity.

Question 2. Are these substitutions adequate, or is the depth proxy in particular too weak to support the per-trade dollar figures? Window volume is two hours of turnover, not size resting at the touch, so it is likely optimistic even after capping.

1.2 Should static arbitrage be documented as a baseline at all?

Table 1: Model minus baseline, net \$/trade, with 95% confidence intervals from a cluster bootstrap over baskets.

baseline	uncapped	cap 25%	cap 10%
median executable size	100	2.5	1.0
random (causal control)	+2.92 [2.27, 3.64]	+0.75 [0.53, 1.03]	+0.67 [0.43, 1.00]
oracle shrinkage	+1.72 [1.14, 2.37]	−0.53 [−0.92, −0.06]	−0.70 [−1.04, −0.35]
always revert	−2.52 [−3.11, −1.89]	−1.26 [−2.00, −0.57]	−0.84 [−1.38, −0.33]
static arbitrage	−21.49	−4.10	−2.84

Two observations on the first three rows. The reversal against the oracle is itself a finding rather than a failure: uncapped, the model wins by \$1.72, and the advantage declines monotonically across three sizing regimes, crossing zero at around 25% participation. That monotonicity makes it a property of liquidity rather than of a chosen threshold: in liquid baskets mean reversion dominates and a single coefficient matches the model, whereas where the model adds more, the market is too thin to fill. The deficit against `always_revert` appears to be trade selection rather than forecast quality: the gross edge is \$3.55 per trade against \$1.12, while the cost is \$21.95 against \$15.72.

My question concerns the last row. My inclination is **not to present static arbitrage** as a baseline strategy at all, because it is not an executable one. It assumes simultaneous fills at quoted prices across every leg, while every other row is charged realistic fees, spreads and a participation cap. With a median executable size of one contract and no order-book history retained by Kalshi, that fill risk cannot be quantified retrospectively. Its 100% win rate is an accounting identity rather than a performance measure.

Question 3. Would you prefer that I (a) drop it from the baseline comparison and report it only in the discussion as an idealised-execution upper bound, (b) retain it in the table but clearly labelled as a bound rather than a competing strategy, or (c) keep it as a full baseline on the grounds that a reader will expect to see it? Should I additionally add a matched-event comparison in which the remaining baselines trade my event set so that costs cancel, which would isolate signal quality as cleanly as the random control already does?

1.3 How should profitability be reported when every strategy loses?

Break-even requires a 33.5¢ move in the basket sum; the mean move is 2.26¢. Every strategy loses net, so the net ranking measures cost avoidance rather than skill, and abstention at \$0 dominates the whole table. This matches the published work: Whelan, *Makers and Takers: The Economics of the Kalshi Prediction Market*, reports an average return of about -20% on Kalshi contracts, with takers at -31.46% against makers at -9.64%, so a negative net result for a taker round trip is the norm on this venue.

Gross of costs, however, the ordering is clean and follows signal quality exactly; see Table 10 in Section 6.4. **The model has the highest gross edge of every strategy that makes a forecast**, and the ranking is monotone in signal sophistication: coin flip \$0.01, unconditional reversion \$1.03, hindsight-fitted shrinkage \$2.05, model \$2.92. Static arbitrage's \$5.56 sits outside that ordering because **it makes no prediction at all**: its return is the arithmetic size of the mispricing under assumed perfect execution. The `random` row is the cleanest comparison, since it trades the same 2,366 events at identical cost (\$33.51 for both), so the entire \$2.91 difference is directional skill with no confound.

Question 4. Is it legitimate to present gross edge as the measure of signal quality, clearly labelled and with net PnL as the headline, its negative value justified by reference to Whelan's measured Kalshi returns above?

1.4 Is there remaining potential in the ladder mechanism?

The ladder result is negative, established four ways. Violations do revert sharply ($+18.5¢ \to +0.07¢$ in one step) but only in the approximately 15% of pairs where a leg actually reprices; the pooled α of 0.99 turned out to be an artifact of the 85% that never traded. Since the model cannot predict which pairs will reprice, it applies the correction indiscriminately, losing about four units of error on unmoved pairs per unit gained on moved ones. The all-pairs ratio is minimised at initialisation and rises under every mover weighting tested.

Two possible causes have different implications. *Structural:* repricing is unpredictable from the current features, in which case more data will not help and the mechanism is closed. *Statistical:* some conditional cells hold only 95–251 observations and the graph ablation was single-seed, in which case the longer run might change the picture.

Question 5. Do you see remaining potential worth pursuing here, for example by adding order-flow or time-to-expiry features aimed specifically at predicting *whether* a pair will reprice, rather than by how much?

Question 6. If not, should I abandon it entirely and present a focused MECE study?

Question 7. Either way, should the ladder work still be documented in the final report? My instinct is yes, since the finding that a pooled coefficient can measure trading frequency rather than market dynamics seems a transferable methodological point; but I recognise that a negative result costs space that could go to MECE, and you originally directed me toward Direction B covering both mechanisms.

2 Research Question

A note on the change of direction. This project was originally scoped as a pairs-trading study: identify pairs of contracts whose prices co-move, estimate the relationship from history, and trade the spread when it diverges. On supervisory advice the scope was redirected to the two structural mechanisms set out below. The distinction is substantive rather than presentational. A pairs-trading spread has no anchor beyond an estimated historical relationship, so a divergence may reflect either a mispricing or a change in the relationship itself, and the two are not separable. The constraints below are exact, known in advance, and enforced at settlement, so a deviation has a defined target rather than an estimated one, and the question of whether it can be forecast is well-posed.

Two no-arbitrage constraints hold on Kalshi by construction.

MECE (sum-to-one). For a set B of mutually exclusive, collectively exhaustive outcomes, the YES prices must sum to \$1 at resolution. The deviation

$$\text{dev}(t) \;=\; \sum_{i \in B} p_i(t) \;-\; 1 \tag{1}$$

is a measurable dislocation.

Ladder (monotonicity). For threshold markets on the same underlying, crossing a higher threshold implies crossing a lower one, so $p(\text{low strike}) \geq p(\text{high strike})$. The quantity

$$\text{gap} \;=\; p(\text{high}) - p(\text{low}) \;>\; 0 \tag{2}$$

is therefore a violation.

Both constraints are routinely violated in observed prices. The project asks a question the existing literature does not: *are these violations forecastable, and if so, is the forecast worth money?*

This is distinct from the detection question that the published work addresses. Detection asks whether an opportunity existed. Forecasting asks whether a dislocation can be anticipated before it appears.

Table 2: Dataset summary.

Venue	Kalshi
Period	2025-05-01 → 2025-09-30 (5 months)
Snapshot interval	2 hours
Observations	1,075,725
Markets live at once	∼700
Distinct markets per month	∼15,000
Validated ladder pairs	2,472,029
MECE basket-snapshots scored	64,717 (val), 41,700 (test)
Ladder pair-snapshots scored	426,061 (val)

3 Data

Splits follow `data_windows.py`, defined by calendar month with no shuffling (Table 3).

Table 3: Chronological train/validation/test split.

split	months	role
train	May, June	weight updates
val	July, August	checkpoint selection, early stopping
test	September	scored once per result, never used to fit anything

Table 4: Per-node features (10 slots, ticker nodes).

slot	feature
0	`last_yes_price` (0–100¢)
1	`yes_vwap`
2	`price_return`
3	`price_std`
4	`window_volume`
5	`net_flow` (signed, −1 to 1)
6	`buy_ratio`
7	`trade_intensity`
8	`time_to_close`
−1	node-type indicator (0 = ticker, 1 = basket hub)

Basket hub nodes carry `sum_cents`, `deviation`, `legs_known`, `legs_total`, `coverage_ratio` and `freshness_ratio` in the same slots, which is why the objective supervises ticker nodes only: slot 0 denotes two different quantities depending on node type.

Data quality, measured and handled. `last_yes_price` falls outside $[0, 100]$¢ in 2.50% of observations (maximum 218); `yes_vwap` is negative in 0.73%; `time_to_close` exceeds two years in 0.33%. All are filtered by `model/data_validation.py` before the active-node set is computed.

What the data does not contain: order-book depth. Kalshi's API exposes the book only for the current moment and does not retain historical state. Retrospective depth analysis therefore requires prospective recording or a commercial archive, neither of which was available for this window. This is why the prediction-market depth literature is almost entirely concerned with Polymarket, whose book is on-chain and permanent, and it is the source of the main execution limitation in Section 8.

4 Architecture

The backbone is a four-stage stack:

1. `TypeSpecificProjection`: separate projections for ticker and hub nodes.
2. `TwoChannelSpatialAttention`: GATv2 over two parallel channels: a MECE hyperedge channel (leg $\leftrightarrow$ basket hub) and a ladder chain channel (directed $\text{leg}_a \rightarrow \text{leg}_b$).
3. `TemporalAttention`: causal, per-node, node-batched with gradient checkpointing.
4. `ResidualForecastHead`: $\hat{p} = p_{\text{now}} + \Delta$, with the final layer zero-initialised.

4.1 Key design decisions, and why they were forced

Residual parameterisation.

$$\hat{p}(t+h) \;=\; p(t) + \Delta_h \tag{3}$$

with the final layer zero-initialised, so that the model begins exactly as the persistence baseline and can only move away from it by reducing loss. Without this, a model trained on a distribution in which roughly 75% of prices do not change converges to persistence and is then reported as "not beating persistence", a statement about the sampling, not the market.

Unconstrained head. The original `MeceOutputHead` used a grouped softmax forcing predicted legs to sum to exactly \$1.00, which makes a predicted dislocation structurally impossible and renders the model's basket deviation identically equal to the naive rule's. `LadderOutputHead`'s $\text{fair}_a \geq \text{fair}_b$ guarantee did the same for monotonicity. Both were replaced: the mechanism now lives in the graph rather than in output constraints.

Causality. `TemporalAttention` builds an unconditional lower-triangular mask, so $h_{\text{final}}[t]$ sees only times $\leq t$. Forecasting $t + h$ from it cannot leak.

Month-bounded chunking. A padded dense (T, N, F) tensor grows with the square of calendar range under Kalshi's node churn: 0.2 GB at one month, 4.4 GB at five months, 34.6 GB at fourteen months, at 285× padding. Per-month build and cache with month-bounded chunks holds peak resident memory flat from one to twelve months.

Standardisation on train only. Raw feature standard deviations span a factor of 37,726,891. Standardisation together with validity filtering took validation loss from 0.374 to 0.0301.

Training configuration. Adam; `embed_dim` 32, `n_heads` 4; chunk length 168 snapshots (about two weeks); horizons $h \in \{1, 2, 3, 6, 12\}$ snapshots (two to twenty-four hours ahead); multi-task loss

$$\mathcal{L} \;=\; w_{\text{node}}\,\mathcal{L}_{\text{node}} \;+\; w_{\text{ladder}}\,\mathcal{L}_{\text{ladder}} \;+\; w_{\text{mece}}\,\mathcal{L}_{\text{mece}}, \tag{4}$$

with early stopping and checkpoint selection on a configurable metric (`-select-on`).

5 Evaluation Protocol

5.1 Notation

The reversion coefficient α. α is the fraction of a dislocation that survives one step: formally, the slope of the least-squares line through the origin fitted to pairs (size now, size later),

$$\alpha^* = \frac{\sum_t x_t\, y_t}{\sum_t x_t^2}, \qquad x_t = \mathrm{dev}(t), \quad y_t = \mathrm{dev}(t+h). \tag{5}$$

Thus $\alpha = 0$ means the dislocation vanishes; $\alpha = 0.5$ that half remains; $\alpha = 1$ that nothing changes, a random walk in levels; and $\alpha > 1$ that the dislocation widens.

α does two different jobs in this project, and the two should not be conflated. *Descriptively*, it measures how fast a class of mispricing corrects. *As a baseline*, the same estimate becomes the shrinkage oracle's forecasting rule.

Pooled versus conditional α. A *pooled* α is fitted over all observations at once; a *conditional* α is fitted separately within a stratum. As Section 6.2 shows, the distinction is not a refinement here but a reversal of the result.

Forecast horizon h. The horizon in snapshots. Since data is two-hourly, $h = 1$ is two hours ahead and $h = 12$ is one day.

Sample size n. The number of supervised positions behind a reported figure.

Shrinkage ratio. The model's MSE divided by the shrinkage oracle's MSE.

5.2 Two baselines, scored on identical positions

Persistence. Predict $\hat{p}(t+h) = p(t)$, scored with the same elementwise loss function as the model.

Optimal shrinkage oracle. Predict $\hat{x}(t+h) = \alpha\, x(t)$ with the single α minimising squared error, **fitted on the split being scored**. This is deliberately an oracle and therefore optimistic: it is the strongest form of the objection "a scalar could do this". A model that merely discovers mean reversion beats persistence without having learned anything about this market; it must beat the oracle to have learned something.

5.3 Stratification

All node-level results are split by whether the price actually moved between t and $t+h$. On flat positions persistence has an MSE of exactly zero and no forecast can beat a tie, so pooling the two regimes hides both.

5.4 PnL evaluation

The taker fee is $0.07\, C\, p(1-p)$ per leg, following Kalshi's published formula, and the maker fee is 0.0175. Per-leg spreads are measured from

`pairwise_monotonicity_taker_side_results_corrected.parquet`

with an explicit and reported fallback chain: exact pair → ticker median → series median → global median. Confidence intervals bootstrap over **baskets** rather than over opportunities, because the same basket sighted at two-hourly intervals is one bet repeated.

Table 5: Control strategies, all trading identical events under identical costs.

control	direction rule	purpose
`random`	coin flip, same entry gate	causal control: only the forecast differs
`oracle_shrink`	$\alpha\,\mathrm{dev}(t)$, α fitted on the scored split	is the model better than a scalar?
`always_revert`	$-\mathrm{sign}\big(\mathrm{dev}(t)\big)$, no model	competing strategy needing no model
`static_arb`	none (no directional bet)	$\lvert\mathrm{dev}(t)\rvert$ held to resolution, one-way costs

Two features of Table 5 are worth making explicit, because they account for figures that appear later. First, `always_revert` and `static_arb` share the same entry gate, $|\mathrm{dev}(t)| > \tau$, which is why both trade the identical 5,493 events in Table 10. Second, `static_arb` has no direction rule at all: its gross edge is booked as $|\mathrm{dev}(t)| \cdot C$, unsigned by construction, since the position is taken on whichever side captures the deviation and then held to settlement. Its 100% win rate is therefore an **arithmetic identity rather than a measure of accuracy**. Because it settles rather than exits, it pays entry fees and one spread crossing per leg, against the two crossings and two fee legs every other strategy pays. That **cost asymmetry, not a superior signal**, is what places it at −$9.10 per trade.

6 Results

6.1 MECE dislocation is forecastable: confirmed on held-out test

Table 6: MECE basket-deviation forecasting, validation and held-out test.

	val ($n = 64{,}717$)	test ($n = 41{,}700$)
model MSE	0.006568	0.00397
persistence	0.008251	0.00485
shrinkage oracle	0.00772 ($\alpha = 0.870$)	0.00466 ($\alpha = 0.917$)
model / oracle	**0.851×**	**0.853×**
model / persistence	0.796×	0.819×

The test split is **harder**: α is closer to 1, so the oracle captured only 3.9% over persistence there against 6.4% on validation. The model's incremental gain over the oracle was nonetheless **14.9% and 14.7%**. The edge is not rediscovered shrinkage: on test there was substantially less shrinkage to rediscover and the margin did not move.

At the **node level on test**, the model attains **0.990× persistence on the 118,559 positions that moved**. The pooled 1.010× includes **343,551 flat positions (74%)** and is an artifact of the sampling rather than a result.

6.2 Ladder violations revert sharply, but only when a leg trades

An earlier test reported the ladder gap as a random walk, with pooled $\alpha \approx 0.99$ in every gap-magnitude bucket. That conclusion was wrong.

A pair in which neither leg reprices satisfies $\mathrm{gap}(t + h) \equiv \mathrm{gap}(t)$ **exactly** (verified, with maximum drift 0.000000) and therefore contributes $\alpha = 1$ by arithmetic. With 75–90% of pairs

in that state, the **pooled coefficient** measures how often Kalshi ladder legs go untraded, not whether the gap reverts.

Table 7: Pooled versus conditional reversion, $h = 1$.

split, $h = 1$	frozen share	pooled α	α on MOVED
val	89.8%	0.9962	0.9273
test	**85.0%**	**0.9928**	**0.8760**

Table 8: Conditional on a reprice (test, $h = 1$).

bucket	n	gap at t	gap at $t + h$	shrinkage ratio
+1¢ to +3¢	110	+1.90¢	−4.85¢	0.726
+3¢ to +8¢	174	+5.13¢	−5.55¢	0.741
≥ +8¢	184	+18.51¢	+0.07¢	0.421

A +18.51¢ violation collapses to +0.07¢ in one two-hour step, and overshoots into negative territory in the smaller buckets. Eighteen cells across two splits and three horizons all agree. The result was independently cross-checked: `diagnose_staleness.py` separately found violation survival collapsing to 33.5% at $h = 1$ once a leg reprices, while the validation +1¢ to +3¢ bucket gives 33.6%.

6.3 The ladder structure cannot be used, tested four ways

Table 9: The ladder mechanism in four roles.

role	result
forecast target, all pairs	1.98× persistence
target, mover-weighted ($W = 3$ / $W = 2$)	1.070× / 1.058×
target, moved-only (oracle-conditioned)	0.958× persistence, 0.971× oracle
graph context for MECE (ablation)	no detectable effect

The all-pairs ratio is **minimised at initialisation** and rises monotonically with every epoch and with every increase in mover weight. The trade is fixed at roughly **four units of error on unmoved pairs per unit gained on moved ones**. As the mover weight tends to zero the model converges to persistence and the ratio approaches 1.0 *from above*. There is no weight at which it crosses.

The ablation, **which removes ladder edges from the graph while holding the loss and all metrics fixed**, changed MECE accuracy by less than the epoch-to-epoch variation of a single run: a best-epoch difference of 0.007 and a mean difference of 0.003 **in the opposite direction**, against a standard deviation of 0.04 across the final ten epochs. A single-seed comparison cannot resolve an effect of that size.

> ***The forecastable object is the* size *of the correction. The unforecastable one is its* timing*, and only the timing was ever missing, because the static arbitrage captures the size by holding.***

6.4 Trading PnL: negative, for three separately quantified reasons

(a) The taker round trip is arithmetically impossible. Kalshi's fee summed over a basket whose legs total 1 gives

$$\sum_i p_i\,(1-p_i) \;=\; 1-\sum_i p_i^2 \;\approx\; 1-\frac{1}{N}, \tag{6}$$

so round-trip fees are approximately

$$2\cdot 0.07\cdot C\cdot\Big(1-\tfrac{1}{N}\Big), \tag{7}$$

nearly independent of leg prices: a fixed toll.

Table 10: Test split, $h = 1$, 100 contracts per leg.

strategy	trades	win %	gross/trade	net/trade
model	2,366	**51.1%**	**$2.92**	−$30.59
`oracle_shrink`	2,405	45.5%	$2.05	−$32.31
`always_revert`	5,493	35.2%	$1.03	−$28.07
`random`	2,366	34.1%	$0.01	−$33.50
`static_arb`	5,493	100%	$5.56	−$9.10

Break-even requires a 33.5¢ move; the basket sum moves 2.26¢ on average and 6.00¢ at the 90th percentile. Even at zero spread cost, fees alone require 6.76¢, so fewer than 10% of opportunities clear costs with a perfect forecast and free execution.

`random` trades the same 2,366 events at identical cost ($33.51 for both) and differs only in direction, so $6,918 against $18 gross is a clean paired comparison with no confound.

(b) Maker execution is worth $29 per trade and still loses. Holding signals fixed and changing only execution moves the result from −$30.59 to −$1.15. Fees fall fourfold, and spreads go from −$26.76 to $0 because a posted entry earns half the spread and a crossed exit pays it back. This is consistent with Whelan's measured Kalshi maker/taker gap (−9.64% against −31.46%). Fills are derived from order flow (a posted bid fills only when `net_flow` × side < 0), not assumed.

(c) The model's signal concentrates in expensive trades. Measured against the no-model reversion rule in the realistic configuration:

Table 11: Gross edge against cost, model versus `always_revert`.

	model	`always_revert`
gross edge/trade	**$3.55**	$1.12
cost/trade	$21.95	**$15.72**
net	−$18.39	−$14.59

The signal is 3.2× stronger; the trades are 40% more expensive. The entry gate fires on predicted dislocation, dislocation scales with leg count, and so do fees and spread crossings. **Signal strength and execution cost share a common cause.** This is a **trade-selection failure, not a forecasting one**.

6.5 The liquidity sweep

Capping trade size at $X\%$ of the thinnest leg-side's window volume removes the thinnest baskets entirely, so a tighter cap leaves a **more liquid surviving population**. Normalising to a per-contract edge:

Table 12: Liquidity sweep: per-contract edge against two baselines.

regime	events dropped	mean size	model − random	model − oracle
uncapped	0%	100.0	+0.0291	+0.0087
cap 25%	24.4%	19.0	+0.0395	+0.0009
cap 10%	45.7%	13.4	+0.0500	−0.0050

Two monotone trends run in opposite directions. **Against the coin flip, +0.029 → +0.040 → +0.050**: the model's directional edge *grows* as the population becomes more liquid. **Against scalar shrinkage, +0.009 → +0.001 → −0.005**: its advantage over a single α *shrinks* monotonically and crosses zero at around 25%.

Both are true, and the combination is interpretable. **In liquid baskets, mean reversion is the dominant dynamic.** The model captures it, hence it beats a coin flip by more there; but a single α captures it too, so the model adds nothing beyond the scalar. Where liquidity is thin, reversion is noisy, the scalar does badly, and the model's richer signal earns its keep, and that is also where the position cannot be filled.

Monotonicity across three regimes makes this a property of *liquidity* rather than of a chosen threshold.

7 The Model's PnL Compared to Baseline Strategies

Forecast accuracy: yes. 0.853× the optimal shrinkage oracle on held-out test, replicating validation's 0.851×, with the margin shown to be independent of how much reversion the period offered.

Directional PnL against the causal control: yes, at every sizing. `random` uses the identical entry rule, identical fill cap and identical execution, with only the direction discarded.

Table 13: Model minus `random`, net $/trade.

sizing	model − random	95% CI
100 contracts/leg	+$2.92	[+2.27, +3.64]
cap 25%	+$0.75	[+0.53, +1.03]
cap 10%	+$0.67	[+0.43, +1.00]

Every interval excludes zero, and the per-contract advantage grows as sizing becomes realistic.

Against competing strategies: no, at realistic size. The model loses to scalar shrinkage (−$0.53 to −$0.70), to the no-model reversion rule, and to the static arbitrage.

Profitable: no, in any configuration with usable capacity. Every strategy tested loses money, and ranking loss-making strategies by which loses least is weak ground: not trading at all scores $0 and beats every row.

The model **forecasts** basket dislocation better than an optimal shrinkage oracle, but does not **trade** better than it once execution size is constrained. Its forecasting advantage is concentrated in baskets too illiquid to fill, and in the liquid baskets where trading is possible a single mean-reversion coefficient matches it.

Defensible contributions

1. A **controlled comparison of two arbitrage mechanisms**, with a measured explanation of why one is forecastable and the other is not.
2. A **forecasting result that survives an oracle baseline** on held-out data, shown to be independent of the period's available reversion.
3. A **quantified anatomy of why a real edge does not trade**, in four independent layers: fee arithmetic, execution mode, the correlation between signal strength and cost, and the concentration of the edge in illiquid baskets.
4. A **methodological result**: a previously published conclusion of this project was overturned by conditioning on whether the price moved at all.

8 Limitations

1. **Order-book depth is unobservable.** Kalshi does not retain historical book state. All PnL is stated per 100 contracts per leg; `window_volume` is used as a participation proxy, which measures two hours of turnover rather than size resting at the touch and is therefore optimistic even after capping.
2. **Spread coverage.** Every MECE spread lookup fell through to a global median borrowed from the ladder pairs file, and spreads are 80% of the taker cost. The conclusion survives because fees alone exceed the 90th-percentile move.
3. **Maker spread netting.** `maker_taker`'s spread nets to \$0 by construction: one static per-leg spread applied to both sides. Exit slippage is unmodelled.
4. **Cluster bootstrap behaviour.** Below roughly ten baskets the cluster bootstrap collapses rather than widening, under-covering in the reassuring direction. Basket counts are printed per row; all reported rows rest on 63–341 baskets.
5. **One test month.** Surviving effects live in the tails, and a single month is thin for tail claims.
6. **The ladder ablation is inconclusive, not negative.** Resolving an effect that small would require repeated seeds.
7. **Small conditional cells.** Ladder violation × MOVED cells hold 95–251 pairs. Credibility rests on eighteen cells agreeing across two splits together with an independent cross-check, not on any single coefficient.
8. **The cost-concentration finding must not be acted on using test data.** Gating on predicted dislocation per unit cost is the obvious next step, but the hypothesis came from inspecting test results and would have to be fitted and selected on validation.

9 Code Inventory

Table 14: Scripts and their purpose.

script	purpose
`train_forecast.py`	forecasting training entry point
`model/forecast_head.py`	residual forecast head
`model/forecast_objective.py`	horizon loss, persistence and shrinkage baselines
`score_forecast.py`	score a checkpoint on a held-out split, once
`evaluate_mece_forecast_pnl.py`	taker PnL, controls, participation cap
`evaluate_mece_maker_pnl.py`	maker PnL with a flow-derived fill model
`evaluate_arbitrage_ladder.py`	ladder PnL with measured spreads
`diagnose_ladder_conditional.py`	reversion conditioned on repricing
`diagnose_ladder_reversion.py`	reversion bucketed by gap magnitude
`diagnose_ladder_cascade.py`	order-flow cascade between adjacent rungs
`diagnose_mask_semantics.py`	what the observation mask means
`diagnose_staleness.py`	violation survival by repricing
`verify_ladder_direction.py`	strike-ordering convention check
`evaluate_violation_prediction.py`	violation emergence signals

Every diagnostic was validated against synthetic data with a planted effect before being run on real data, to confirm that it could detect a positive result before a null one was interpreted.